Technical Test: GenAI DevOps Intern

Overview

This is a take-home DevOps test. You take a small FastAPI data-analytics service and make it production-ready across two host classes through one consistent pipeline. We are testing operational judgement — architecture, observability, and how your design responds when requirements change mid-flight.

The task

In `app/` you will find a small FastAPI service that exposes a natural-language query endpoint over tabular data — a user asks a question, the service returns a cited numeric answer plus a chart spec.

Take it to deployable on two host classes (AWS EC2 and `localhost`) through one consistent pipeline. Make it observable, recoverable, and ready for the next engineer. You will not be asked to present your work in an interview call — the submission is read on its own.

Pick a domain

You pick one — the rest of the requirements are identical.

- **Finance.** Multi-source price and corporate-action data. Sources disagree on the same (`ticker, day`). Numbers must round to the cent.
- **Health.** Public-health surveillance counts per district plus weather covariates. Sources use different reporting calendars and naming conventions.
- **Environment.** Multi-source weather observations from stations and APIs. Sources differ on schema, units, and sampling cadence.

A small starter dataset per domain is in `data/`. Extend it, bring public data, or generate synthetic — justify briefly in `ARCHITECTURE.md`.

Context to design against

- **Brownfield, not blank slate.** Decide what to fix and what to leave alone — tell us why.
- **Two targets, one image.** `localhost` is the live target you must actually run end-to-end. AWS EC2 is a documented design — the deploy script supports the `aws` target and the per-host `.env` exists, but you do not need to execute it against real AWS. Same image; only `.env` and host volumes differ.
- **Constraints will move.** Expect a curveball on day 2 — a requirement change emailed to you. Architect for it; we are watching how the system absorbs it.
- **An on-call engineer owns this after you.**

Timeframe

3 days. Document assumptions as you go.

AI-assisted coding is permitted and expected. Include `AI_USAGE.md` (tool, where, what you accepted / modified / rejected, how you verified). *Optional:* share session-recording excerpts.

No paid services required. The bundled mock LLM provider runs offline; if you want a real LLM, Ollama runs locally for free. GitHub Actions free tier, GHCR public images, locally-run Grafana / Prometheus / Loki, and free uptime monitors (UptimeRobot, Healthchecks.io) all suffice. No AWS account needed.

What you deliver

1. **Containerised image** — deterministic install, no first-request downloads, image tag is the git SHA.
2. **Compose** for local-and-server use, fronted by a reverse proxy with HTTPS.
3. **One deploy script** targeting `aws` or `localhost` via per-host `.env` overrides — validates env, polls health, rolls back on failure. `localhost` must actually run; the `aws` path can stub the AWS-specific calls and be documented in the runbook. Include the systemd unit.
4. **CI** — lint, test, build, push the image on `main`; deploy to a staging target.

5. **Architecture document** (`ARCHITECTURE.md`) defending three choices: what the LLM sees (raw rows vs metadata plus deterministic summaries); where deterministic compute lives and how a query is reproducible; what is cacheable, at which layer, with what invalidation rule. One diagram.
6. **Data-quality handling** — when sources disagree on the same record, the tool surfaces the disagreement to the user instead of picking silently. Document the rule.
7. **Observability** — correlation IDs propagated end-to-end (trace, query, dataset, role); structured logs; a `/health` that says more than 200; one dashboard that lets an operator pivot from "something is slow" to the exact stage; one alert that would page on a real outage.
8. **One surprise.** Pick one failure mode you think we would miss and address it concretely. One thing, not a survey.

Deliverables

Git repo (preferred) or zip:

1. **`README.md`** — setup, design decisions, what you left as-is in the inherited code and why; what you are least confident about; what you would add with another 8 hours. An honest answer beats a padded one.
2. **`ARCHITECTURE.md`** — the three choices above, with one diagram.
3. **`RUNBOOK.md`** — operator-facing: deploy, roll back, `/health` field semantics, where logs live, who to wake at 3 a.m.
4. **`AI_USAGE.md`**.
5. A short recording or annotated screenshots: a deploy succeeding, the day-2 curveball absorbed, the dashboard with traffic, one alert firing on a forced failure.

Evaluation

We weight, in order: (1) does it run end-to-end on a fresh machine; (2) did the architecture absorb the day-2 curveball without a rewrite; (3) is the observability layer rich enough to debug from telemetry alone; (4) is the data-quality handling defensible; (5) is your "one surprise" thoughtful.

We do not grade UI polish, vendor choice, or model accuracy of the stand-in app.

Contact

adish@artpark.in